

Escribir skills y agentes para SDD

Lección 3 – Spec-Driven Development

Encodear el playbook del equipo para que toda sesión de agente arranque con él.

Qué vamos a ver hoy

- 1 Spot review** – un equipo sorteado presenta la tarea de la Lección 2
- 2 ¿Qué es un skill?** – y por qué encodear el playbook una vez, no cada sesión
- 3 La caja de herramientas** – qué recurso resuelve qué problema
- 4 Anatomía de un skill** – SKILL.md, triggers, divulgación progresiva y cómo probarlo
- 5 Skills SDD en la práctica** – el pipeline, encodeado
- 6 Subagents, hooks y rules** – delegar, garantizar, fijar; y una demo en vivo
- 7 Briefing de la tarea** – un toolkit SDD (en equipo)

Al final de esta clase podés...

Explicar qué es un skill y cómo lo carga el agente – en Claude Code, Codex y Grok Build

Escribir un skill SDD con un trigger que dispara, y probarlo

Definir un subagent con tools restringidas, y decidir qué delegar en paralelo y qué en serie

Garantizar una práctica SDD con un hook que veta, y fijar las demás en rules

Elegir el recurso correcto – skill, rule, subagent o hook – según el problema

Spot review

Tarea de la Lección 2 – tmux con SSH nativo, un equipo, en vivo

Qué estamos mirando

Un **equipo sorteado** nos muestra su análisis y spec de **tmux con SSH nativo**.



Calidad del descubrimiento

¿Mapearon el camino real de spawn, la tabla de comandos y la capa `compat/` – con archivos y funciones nombradas?



Disciplina de alcance

Superficie de cambio acotada, fuera-de-alcance explícito, y el **límite solo-Linux** escrito como invariante.



Cobertura de VCs

FRs + VCs que referencian archivos reales; "los builds no-Linux siguen compilando" como invariante chequeable.

La vara era **análisis + spec**, no código C. ¿Resistieron las ganas de implementar?

¿Qué es un skill?

Y por qué dejar de reenseñarle el mismo playbook al agente cada sesión

¿Qué es un skill?

Un **skill** es **conocimiento procedimental empaquetado**: instrucciones para una tarea, más una **metadata** (name, description) que el agente registra al iniciar la sesión. El modelo las **carga en su contexto, a demanda**, cuando decide que la tarea lo requiere.

Cómo funciona, dentro del loop del agente

1 • Registro al iniciar la sesión

El runtime lee la metadata de cada skill y la pone en el contexto: **~100 tokens por skill**. El cuerpo todavía no está.



2 • Selección en PLANIFICAR

El modelo compara el pedido contra las **description**. Elegir un skill es una decisión más del loop, como elegir qué archivo leer.



3 • Carga en ACTUAR → LEER

Una tool call trae el cuerpo a la ventana; si el cuerpo lo pide, también plantillas o scripts. Desde ahí, son instrucciones en el contexto.

No es un prompt

no se pierde al cerrar la sesión

No es una rule

su cuerpo no está en cada sesión

No es una tool

no le agrega acciones al agente

No es entrenar

el modelo no cambia: lee texto

Lo elige el modelo, así que un skill **persuade**: todo depende de la **description**.

El impuesto de reexplicar

Cada sesión pegás el mismo contexto: *"escribí la spec con FRs en Dado/Cuando/Entonces, cada FR necesita un VC, mantené el alcance acotado, listá qué no tiene que cambiar..."*



Sin encodear

Se retipea cada vez. Deriva entre personas. Se recuerda a medias. Al que recién entra no le llega nunca.



Como skill

Escrito una vez. Cargado automáticamente. Idéntico para todos. Auditable y mejorable en el control de versiones.

Un **skill** es tu flujo de trabajo, escrito donde el agente lo lee – para que toda sesión arranque desde el mismo playbook.

Skills vs. rules vs. prompts sueltos

Tres formas de darle instrucciones al agente – distinta vida útil, distinto trabajo.

	Prompt suelto	Skill	Rule
Vive en	el chat	un archivo SKILL.md	la config del proyecto
Vida útil	este mensaje	se invoca cuando aplica	siempre activa
Sirve para	esta tarea puntual	un flujo que repetís	restricciones que siempre valen
Ejemplo	"renombrá esta variable"	"escribí una spec SDD"	"siempre corré los tests antes de commitear"

Las rules son **restricciones** (los guardrails). Los skills son **flujos** (los playbooks). Los prompts son **por esta vez**.

La caja de herramientas

Todo lo que le podés dar a un agente de código

El panorama de recursos

Más allá del prompt, a un agente se lo puede extender de varias formas. Sabé para qué sirve cada una.

Skills

Flujos reusables en **SKILL.md** – pasos, plantillas, anti-patrones. Se cargan cuando la tarea matchea.

Rules

Restricciones persistentes (**CLAUDE.md**, **AGENTS.md**) aplicadas en cada sesión.

Subagents

Ayudantes delegados para una tarea puntual – explorar, revisar, implementar – con contexto propio.

Hooks

Scripts disparados por eventos alrededor de las acciones del agente – chequeos pre-commit, spec-antes-que-código, logs de auditoría.

Tools

Las acciones incorporadas – leer/escribir archivos, correr comandos, buscar. Lo que el agente realmente *puede hacer*.

MCP y plugins

Puentes a sistemas externos – Jira, bases de datos, docs. *Lección 4*.

¿Qué recurso resuelve qué problema?

"Explico siempre el mismo **flujo**" → un **skill**

"Esta **restricción** tiene que valer siempre" → una **rule**

"Esta **subtarea puntual** me ensucia el contexto" → un **subagent**

"Este chequeo tiene que correr **pase lo que pase**" → un **hook**

Skill, rule y subagent = *persuadís* al modelo. Hook = **garantizás** el comportamiento.

CHECKPOINT

¿Qué recurso usás?

El equipo te pide cuatro cosas para su agente. ¿Qué recurso resuelve cada una?

Que nadie pueda commitear con la suite de tests en rojo.



Hook

Tiene que valer pase lo que pase: hace falta garantizar, no persuadir.

Que el agente sepa, en toda sesión, que los tests se corren con `pytest -q`.



Rule / CLAUDE .md

Dato siempre verdadero y siempre necesario: se carga en cada sesión.

Que cada spec salga con FRs en Dado/Cuando/Entonces y un VC por requerimiento.



Skill

Un flujo que repetís, cargado solo cuando la tarea matchea.

Mapear auth, storage y la capa CLI sin llenar el contexto de la sesión principal.



Subagents

Firewall de contexto, y tres tareas independientes que corren en paralelo.



Dato siempre activo → CLAUDE .md · flujo → skill · subtarea ruidosa → subagent · **garantía** → hook

Anatomía de un skill

Qué va en un SKILL.md – y por qué la description es lo que más importa

La estructura de SKILL.md

Un SKILL.md tiene dos partes:

Front-matter – el bloque YAML entre las dos líneas `---` del principio. Son datos *sobre* el skill, no instrucciones: lo que el agente registra al iniciar la sesión.

- **name** – cómo se llama
- **description** – *frases disparadoras*; cuándo auto-aplicarlo

Cuerpo – el markdown que sigue. Es lo que se carga cuando el skill dispara.

- **instrucciones, plantillas y anti-patrones**

La **description** es la línea más importante: es cómo el agente decide cargar el skill.

```
---                                # front-matter
name: write-spec
description: Usar cuando el usuario
  pide escribir una spec SDD de un
  feature. Produce FRs, BRs, NFRs y VCs.
---                                # cuerpo ↓

## Pasos
1. Reformular propósito y alcance.
2. Listar actores.
3. Redactar FRs en Dado/Cuando/Entonces.
4. Agregar un VC por cada FR y BR.
5. Marcar preguntas sin resolver.

## Plantilla
FR-N: <comportamiento atómico>
  Dado ... Cuando ... Entonces ...
VC-N: <chequeo observable>

## Anti-patrones
- FR sin VC
- NFR vago ("rápido")
```

Triggers: la description es una promesa

El agente lee la **description** de cada skill y carga el que matchea la tarea. Escríbala como un trigger, no como un título.

✗ No va a disparar de forma confiable

description: Ayuda con specs

Demasiado vago – el agente no puede saber cuándo aplica.

✓ Dispara en la tarea correcta

description: Usar cuando el usuario pide escribir, redactar o revisar una spec SDD de un feature.

Nombra la situación y el fraseo que debería invocarlo.

Nombrá la **situación** ("usar cuando el usuario pide..."), no solo el tema. Los triggers son el contrato entre la intención y el flujo.

Divulgación progresiva: lo que cuesta un skill

Un skill no entra entero a la ventana. Entra **de a capas**, y solo cuando hace falta.

1 • Metadata ~100 tokens

name + description. **Siempre** en la ventana: es lo que el agente mira para decidir.

2 • Cuerpo < 5k tokens

Pasos, plantilla, anti-patronos. Entra **solo cuando la tarea matchea** la description.

3 • Archivos adjuntos a demanda

Plantillas largas, scripts, ejemplos junto al SKILL.md. Se leen **solo si el flujo los pide**.

La barra de la Lección 2, con seis skills instalados: los del pipeline SDD (write-spec, review-spec, ... freeze-spec)

Sesión recién abierta

system prompt +
tools

rules del
repo

6x100 pedido

libre

Pedís una spec

system prompt +
tools

rules del
repo

6x100 pedido

libre

Sin skills: los seis en CLAUDE.md

system prompt +
tools

rules del
repo

los seis cuerpos

pedido

libre

■ **6x100** : la metadata de los seis skills – name + description de cada uno, ~100 tokens c/u.

■ el cuerpo de write-spec, cargado porque la tarea lo pidió: uno solo, no los seis.

Como skills, los seis cuestan **~600 tokens**. En las rules, **sus seis cuerpos**, en cada sesión.

Skills personales vs. de proyecto

Dónde vive un skill decide quién lo recibe.

Skill personal

En tu configuración de usuario. Te sigue a vos por todos los proyectos. Tus hábitos, tus atajos.

p. ej. tu formato personal de mensajes de commit.

Skill de proyecto

Commiteado al repo. Lo recibe todo el equipo. El playbook compartido, versionado junto al código.

p. ej. el `write-spec` y el `review-spec` del equipo.

El conocimiento del equipo va en el **repo** – revisado en PRs, mejorado con el tiempo, idéntico para todos.

Un skill, tres agentes

El mismo **SKILL.md**, y los tres lo eligen solos por la **description**.

Front-matter: el bloque YAML entre `---` al principio del **SKILL.md**, con los datos del skill (**name**, **description**, ...). Claude Code y Grok aceptan campos propios ahí; Codex los pone en un archivo aparte. Son las "extensiones" de la tabla.

	Claude Code	Codex CLI (OpenAI)	Grok Build (xAI)
Origen	lo crea Anthropic (oct. 2025) y lo abre como estándar: agentskills.io (dic. 2025)	adopta el estándar (dic. 2025)	lee SKILL.md , pero no figura entre los adoptantes del estándar
Dónde lo busca	<code>.claude/skills/</code> · <code>~/claude/skills/</code>	<code>.agents/skills/</code> · <code>~/agents/skills/</code>	<code>.grok/skills/</code> , y también <code>.claude/skills/</code>
Invocarlo a mano	<code>/write-spec</code>	<code>\$write-spec</code> , o elegir en <code>/skills</code>	<code>/write-spec</code>
Qué precarga	name + description ; todas juntas, hasta ~1% de la ventana	name + description + path ; hasta ~2% de la ventana	no documentado
Extensiones	en el front-matter: context: <code>fork</code> , <code>allowed-tools</code> , <code>model...</code>	aparte, en <code>agents/openai.yaml</code>	when_to_use , paths ; ignora <code>allowed-tools</code> y <code>model</code>

Viaja: **name** + **description** + el cuerpo. Con eso, el mismo archivo carga en los tres.

No viaja: las extensiones. Un **context:** `fork` de Claude Code, en otro agente, se ignora.

CHECKPOINT

¿Por qué no dispara?

Escribiste un skill **review-spec** con un checklist impecable. Le decís al agente "*revisá esta spec*" y la revisa a su manera, sin cargar el skill. El front-matter dice **description: Checklist de specs. ¿Dónde está el bug?**

A En los **pasos**: el checklist es demasiado largo para que lo siga. ❌

El cuerpo ni se llegó a leer: el agente decide si carga el skill mirando solo la description.

B En la **description**: nombra un tema, no la situación que lo invoca. ✅

La description es el trigger. "Usar cuando el usuario pide revisar o habilitar una spec SDD" sí dispara.

C En la **ubicación**: hay que pegar el checklist en **CLAUDE.md**. ❌

Inflás cada sesión con un flujo que se usa a veces. Un flujo va en un skill, cargado a demanda.

D En el **tipo de recurso**: tendría que ser un hook. ❌

Un hook se dispara por eventos (antes de un tool, al iniciar sesión), no por lo que pide el usuario.

💡 ¿No dispara? **El bug es la description**, no tu flujo.

Probar un skill: ¿dispara cuando tiene que?

Un skill sin probar es una spec sin VCs. Antes de darlo por hecho, armá sus casos **como si fueran criterios de verificación**.

Tiene que disparar

Frases naturales, **sin nombrar el skill**:

¿Esta spec está lista para planificar?

Revisame specs/export-csv.md antes de arrancar.

Fijate si le falta algo a esta spec.

No tiene que disparar

Frases **parecidas** que piden otra cosa:

Revisá este PR.

Escribime la spec de export a CSV.

¿Qué hace la función parse_spec()?

Dispara de menos

La description es un tema, no una situación. Sumale el fraseo real: "lista", "le falta algo", "revisame".

Dispara de más

Es demasiado amplia. Acotala: "una spec SDD", no "revisar algo". Decí cuándo *no* usarlo.

Cómo lo ves

El agente avisa qué skill carga. /skills lista los instalados; en Claude Code, /context muestra cuánto ocupan.

Una sesión nueva por caso: si el skill ya está cargado de antes, la prueba no prueba nada.

Skills SDD en la práctica

El pipeline que corriste a mano ya son seis skills y un subagent

El pipeline, encodeado

El pipeline de las Lecciones 1-2, paso por paso. Cada paso que corriste a mano ya tiene su recurso.



Descubrir no es un skill: es un **subagent**. Lo que importa ahí no es el flujo, es que el barro no cruce.

Cómo tres de ellos imponen disciplina

write-spec – cada requerimiento recibe un VC

La plantilla aparea cada FR/BR con un criterio de verificación. *No podés* producir una spec con un requerimiento inverificable – el skill no deja que esa forma exista.

review-spec – un gate de aptitud, no una impresión

Corre un checklist fijo: ¿FRs atómicos? ¿caminos de falla? ¿preguntas abiertas resueltas? Devuelve **listo** o **huecos** – el mismo gate siempre, para todos los autores.

implement-spec-plan – verifica por iteración

Despacha un subagent por iteración, exige el contrato de artefactos de la Lección 1 (`CONTEXT.md`, `iterations/NN`) y chequea sus VCs – y, en brownfield, la suite existente – antes de seguir. El gate deja de depender de que te acuerdes.

CHECKPOINT

¿Cómo encodeás Descubrir?

Tu equipo ya tiene `write-spec` y `review-spec`. Ahora quiere encodear el paso 0 de brownfield: *mapear el código antes de especificar*, como hicieron a mano con `tmux`. **¿Con qué recurso?**

A Un **skill discover** con los pasos de exploración, corrido en la sesión principal. ❌

El flujo queda bien escrito, pero las 40 lecturas caen en la ventana que después escribe la spec. El barro cruza.

B Un **subagent explore** con **tools: Read, Grep, Glob** que devuelve notas con formato fijo. ✅

Explora en su propia ventana y solo devuelve la conclusión. Y el solo lectura no depende del brief: no tiene con qué editar.

C Una línea en `CLAUDE.md`: "*Siempre explorá el código antes de especificar*". ❌

Persuade, y no dice cómo ni qué devolver. Cada sesión carga una instrucción que el modelo puede saltar.

D Un **hook SessionStart** que corre un grep del repo y lo inyecta en el contexto. ❌

Determinístico, pero no sabe qué buscar: llena la ventana de ruido antes de saber cuál es el cambio.



Lo que define a Descubrir no es el flujo, es el aislamiento. **Trabajo ruidoso → subagent.**

Subagents, hooks y rules

Delegar para enfocarse – hooks para garantizar – rules para fijar

Subagents: ya los usaron, hoy los escriben

Un subagent es un agente con **su propia context window**, al que le das un trabajo enfocado. Devuelve la conclusión, no el camino.

LECCIÓN 2 • DESCUBRIR

Firewall de contexto

El explorador lee 40 archivos en su ventana y devuelve notas de 10 líneas. A la sesión principal llega la señal, nunca el barro.

LECCIÓN 2 • EN PARALELO

Abanico para leer

Flujo, historia y tests: tres subagents al mismo tiempo, cada uno enfocado. El principal consolida las tres conclusiones.

LECCIÓN 1 • IMPLEMENTAR

Serie para escribir

Un subagent por iteración, uno detrás del otro: cada uno arranca del commit anterior y de los artefactos en disco.

El paralelismo solo es seguro con tareas **independientes**. Dos agentes escribiendo el mismo archivo es una race condition, no una aceleración.

Hasta acá los lanzabas con un prompt. Hoy los **definís en un archivo**: brief, tools y formato de salida, versionados en el repo.

Ejemplos: orquestar subagents

El agente principal es el **orquestador** – parte el trabajo, lanza ayudantes, y fusiona lo que devuelven.

Descubrimiento en paralelo (brownfield)

Tres exploradores solo-lectura, una fusión. Alimenta directo a `write-spec`.

Lanzá 3 agentes de exploración – uno para el camino de spawn, uno para la tabla de comandos, uno para `compat/`. Cada uno devuelve solo notas. Después fusionalas en un documento de exploración.

Panel de revisión independiente

Un revisor que no escribió la spec, juzgando contra criterios fijos.

Corré `review-spec` como subagent sobre esta spec. Devolvé listo/huecos con razones – no edites la spec.

Skill + subagent, encadenados

Orquestar el pipeline: cada paso es un skill, corrido por un agente enfocado.

`explorar (subagent) → write-spec (skill) → review-spec (subagent) → implement-spec-plan (skill)`, una iteración por agente nuevo.

Anatomía de un subagent

Un subagent es un archivo markdown, igual que un skill. En Claude Code vive en `.claude/agents/<name>.md`.

- **name** – cómo se lo invoca
- **description** – *cuándo delegarle*; es un trigger, como en un skill
- **tools** – lo que **puede** hacer. Sin Edit ni Write, no edita.
- **cuerpo** – el brief: la tarea, las restricciones y el **formato de salida**

"Solo explorar, sin editar" en el brief *persuade*. `tools: Read, Grep, Glob` lo **garantiza**.

```
---
name: explore
description: Usar para mapear código
             desconocido antes de especificar un
             cambio. Solo lectura; devuelve notas.
tools: Read, Grep, Glob
---

Solo explorar, sin editar.

Tu tarea: <la pregunta de descubrimiento>

Devolvé solo notas, con tres secciones:
1. Módulos tocados – path:línea
2. Interfaces usadas – dónde se definen
3. Riesgos – callers, convenciones,
   estado de la suite

Si no encontrás algo, decilo.
No lo supongas.
```

 CHECKPOINT

¿En paralelo o en serie?

El orquestador tiene cuatro trabajos para delegar a subagents. ¿Cuáles puede lanzar en paralelo?

Mapear auth, storage y la capa CLI antes de escribir la spec.

Paralelo

Tres lecturas independientes: nadie edita.

Implementar las iteraciones 2 y 3 del plan.

Serie

La 3 arranca del commit y los artefactos de la 2 (Lección 1).

Revisar un diff en seguridad, performance y cobertura de VCs.

Paralelo

Tres revisores de solo lectura; ninguno depende de otro.

Refactorizar dos funciones de `billing.py`, una por subagent.

Serie

Mismo archivo: es una race condition, no una aceleración.



Abanico para **leer**. Serie para **escribir**.

Hooks: guardrails que no dependen de la buena voluntad

Un hook es un script que se dispara ante un **evento** – determinísticamente, piense o no el modelo en hacerlo.

Chequeos pre-commit

Corré tests / lint antes de que entre un commit. Suite roja ⇒ commit bloqueado.

Spec antes que código

Bloqueá ediciones al fuente cuando no existe una spec revisada del cambio.

Traza de auditoría

Logueá cada acción de tool, para que un cambio pueda rastrearse hasta su spec.

Puntos de disparo: **antes** de que corra un tool (vetarlo), **después** de que corra (reaccionar/loguear), o al **iniciar la sesión** (sembrar contexto).

Anatomía de un hook

Tres piezas: dónde se registra, qué recibe y cómo decide. El ejemplo es un gate de regresión.

1 • Se registra • `.claude/settings.json`

```
"PreToolUse": [{  
  "matcher": "Bash",  
  "hooks": [{ "type": "command",  
    "command": ".claude/hooks/regression-gate.sh" }]  
}]
```

2 • Recibe el evento • JSON por stdin

```
{ "hook_event_name": "PreToolUse",  
  "tool_name": "Bash",  
  "tool_input": { "command": "git commit -m 'feat: ...'" } }
```

3 • Decide • `regression-gate.sh`, recortado

```
CMD=$(jq -r '.tool_input.command')  
echo "$CMD" | grep -q 'git commit' || exit 0  
npm test --silent && exit 0  
echo "Suite roja: commit bloqueado" >&2  
exit 2
```

exit 0 la acción pasa
exit 2 **veta la acción**; lo que escribiste en stderr le llega al agente
otro error no bloqueante: la acción pasa igual

El **matcher** filtra por tool (**Bash**), no por comando: que sea un **git commit** lo decide el script.

Ejemplos: tres hooks SDD

Condensados de la anatomía: mismo contrato, tres eventos. **Exit 2** bloquea la acción.

Gate de regresión – antes del commit

```
# PreToolUse · matcher Bash · si el comando es git commit
pytest -q || { echo "Suite roja – commit bloqueado" >&2; exit 2; }
```

La seguridad ante regresiones de la Lección 2, hecha política.

Spec antes que código – antes de editar

```
# PreToolUse · matcher Edit|Write · si el archivo está bajo src/
ls specs/*.md >/dev/null 2>&1 || { echo "No hay spec para este cambio" >&2; exit 2; }
```

La regla central de SDD, hecha cumplir: sin spec no se toca el fuente.

Traza de auditoría – después de cualquier tool

```
# PostToolUse · sin matcher: todos los tools
echo "$(date -u) $TOOL $FILE" >> .sdd/audit.log
```

Cada acción logueada: un cambio rastrea hasta su spec. No bloquea.

Skills y hooks ajenos: código con tus permisos

Un skill o un hook que copiás de internet, o que llega en un MR, corre **con tu usuario, en tu máquina, con tus credenciales**.

Hook

Es un script que corre solo, ante cada evento. Un `PostToolUse` sin matcher se ejecuta **después de cada tool**: puede leer `.env`, mandar datos afuera o borrar archivos, y no te enterás.

Skill

Son instrucciones que el modelo **obedece**, y pueden traer scripts. Un paso escondido – "antes de empezar, corré `scripts/setup.sh`" – es una inyección de prompt con tu sesión.

Leé antes de instalar

el `SKILL.md`, los scripts y cada `command` del `settings.json`.

Revisalos en el MR

`.claude/` es código: un cambio en un hook se revisa como cualquier otro.

Confía a propósito

Codex (`/hooks`) y Grok Build (`/hooks-trust`) no corren hooks de proyecto sin que los apruebes.

Nada sensible en stderr

lo que el hook escribe ahí lo lee el agente, y queda en la transcripción.

El mismo poder que hace que un hook **garantice** es el que lo hace **peligroso**.

Rules: las restricciones siempre activas

Los skills se disparan cuando aplican. Las **rules** se aplican a *todas* las sesiones – son las órdenes permanentes.

Dónde viven

CLAUDE.md, AGENTS.md, .claude/rules/ – commiteadas al repo, versionadas con el código. Las vimos agente por agente en la Lección 2.

Qué contienen

Convenciones, límites, "nunca toques X", "los mensajes de commit referencian la iteración de la spec".

AGENTS.md – Mantené siempre los cambios dentro de los archivos en alcance de la spec. Cada commit referencia su spec + iteración. Nunca edites generated/.

Cada recurso, en cada agente

Mismo modelo mental en los tres. Cambian el path, el formato y cómo se veta.

	Claude Code	Codex CLI	Grok Build
Rules	CLAUDE.md jerárquico + .claude/rules/; si no hay CLAUDE.md, lee AGENTS.md	AGENTS.md jerárquico, tope 32 KiB entre todos	AGENTS.md o CLAUDE.md + .grok/rules/, sin tope
Skills	.claude/skills/<name>/SKILL.md	.agents/skills/<name>/SKILL.md	.grok/skills/; también carga los de .claude/skills/
Subagents	.claude/agents/*.md; tools restringe qué puede hacer	.codex/agents/*.toml; sandbox_mode = "read-only"	.grok/agents/; el explore de fábrica es solo lectura
Hooks	.claude/settings.json	.codex/hooks.json o config.toml	.grok/hooks/*.json; también lee .claude/settings.json
Veto	exit 2, o JSON deny	exit 2, o JSON deny; hay que confiar el hook	exit 2, solo en PreToolUse; hay que confiar el hook

SKILL.md es un formato abierto: con name y description, **el mismo archivo carga en los tres**. Lo que más cambia entre agentes es el subagent.

CLAUDE.md – el brief permanente del proyecto

El archivo de reglas que algunos agentes leen **automáticamente al iniciar la sesión** – CLAUDE.md (Claude Code), AGENTS.md (Codex, Grok Build). Misma idea, distintos nombres.

Responde la pregunta que el agente se hace siempre:
"¿Qué necesito saber de este repo antes de tocarlo?"

Se carga en **cada** sesión, así que cada línea cuesta contexto. Mantenelo corto, con señal alta, y verdadero.

```
# CLAUDE.md

## Proyecto
gcsgrep – grep para objetos de GCS.

## Comandos
- Test:  pytest -q
- Lint:  ruff check .

## Convenciones
- Las specs van en specs/, los planes en specs/*-plan.md
- Cada commit referencia su spec + iteración
- Nunca editar generated/

## SDD
- Sin edición de fuente sin una spec revisada (ver write-spec)
```

CLAUDE .md – buenas prácticas



Sí

- **Corto y escaneable** – bullets, no prosa. Se lee en cada sesión.
- **Comandos de build/test/lint** – lo primero que el agente necesita.
- **Convenciones y límites** – "nunca toques X", formato de commit.
- **Apuntá a los skills** – linkeá el playbook, no lo pegues adentro.
- **Commiteado al repo** – revisado en PRs, evoluciona con el código.



No

- **Volcar el manual entero** – infla cada sesión, entierra la señal.
- **Pegar un flujo completo** – eso es un *skill*, cargado a demanda.
- **Dejarlo pudrir** – un comando desactualizado es peor que ninguno.
- **Guardar secretos** – está commiteado y compartido.

Regla práctica: **siempre-verdadero y siempre-necesario** → CLAUDE .md. Un **flujo** → un skill. Una **garantía dura** → un hook.

CLAUDE .md no son tus specs

Dos tipos de documento, dos trabajos distintos. No los colapses.



Artefactos SDD (Lecciones 1-2)

Specs, `-plan.md`, notas de exploración, context capsules.

Sobre: *qué construir y cómo se verifica* – el contrato.

Alcance: un feature / un cambio.

Vida útil: crecen por cambio; se congelan al entregar.

Se cargan: a demanda, en el trabajo que los necesita.



CLAUDE .md (hoy)

Un brief permanente y corto en la raíz del repo.

Sobre: *cómo trabajar* en este repo – comandos, convenciones, límites.

Alcance: el proyecto entero.

Vida útil: estable; cambia poco.

Se carga: automáticamente, en cada sesión.

Las specs son la **fuentes de verdad del trabajo**. CLAUDE .md es el **manual de operación del repo**. Debería *apuntar* a dónde viven las specs – nunca contenerlas.

CHECKPOINT

Lo dice CLAUDE.md, y no alcanza

Tu CLAUDE.md dice *"Siempre corré los tests antes de commitear"*. Aun así, esta semana entraron dos commits con la suite en rojo. ¿Qué cambiás?

A Reforzar la instrucción: mayúsculas, "IMPORTANTE", repetirla dos veces. ❌
Sigue siendo persuasión: el modelo puede ignorarla o reinterpretarla, con o sin mayúsculas.

C Un **skill** run-tests con los pasos para correr la suite. ❌
Un skill se carga cuando la tarea matchea: nada garantiza que se invoque antes de cada commit.

B Un **hook** PreToolUse sobre git commit que corre la suite y sale con código 2 si falla. ✅
Corre determinísticamente ante el evento: suite roja ⇒ commit bloqueado, piense lo que piense el modelo.

D Un **subagent** revisor que mire cada commit antes de hacerlo. ❌
Mejor revisión, misma garantía: sigue dependiendo de que el agente decida invocarlo.

💡 Skill, rule y subagent **persuaden**. Un hook **garantiza**.

Demo: el toolkit en acción

El toolkit para ADRs del ejemplo guiado, instalado en un repo real – p. ej. el `gcsgrep` de un equipo. Muestra la forma, no la tarea.

1 · El skill dispara solo

Leemos los objetos con `google-cloud-storage` y no con `gcsfs`. Dejemos registrado por qué.

Qué mirar: el agente carga `write-adr` sin que lo nombres, y el ADR sale con dos opciones y consecuencias negativas.

Evidencia: la línea donde carga el skill.

2 · El subagent aísla

¿Ya decidimos algo sobre cómo autenticamos contra GCS?

Qué mirar: se lanza `adr-explorer`; a la sesión principal vuelve una lista corta con evidencia, no las lecturas.

Evidencia: la lista, y `/context` antes y después.

3 · El hook veta

Sumá `tenacity` para reintentar las lecturas, y `commiteá`.

Qué mirar: `adr-for-deps` veta el commit con `exit 2`: cambia una dependencia y no hay ADR. El agente lee el `stderr` y lo escribe.

Evidencia: el mensaje de bloqueo y el commit que no existe.

Tres piezas, tres evidencias: la misma **forma** que pide la tarea, sobre otra práctica.

Conceptos clave – repaso

-  **Skill** – conocimiento procedimental que el modelo carga a demanda; el mismo **SKILL.md** sirve en los tres agentes
-  **Triggers** – la description decide cuándo dispara; se prueba con casos que disparan y casos que no
-  **Divulgación progresiva** – metadata siempre, cuerpo a demanda: seis skills cuestan seis descriptions
-  **Subagents** – firewall de contexto; tools restringidas; abanico para leer, serie para escribir
-  **Rules / CLAUDE.md** – datos y restricciones siempre activos, cargados en cada sesión
-  **Hooks** – guardrails por evento que *garantizan* con `exit 2`; código con tus permisos

Herramienta correcta, vida útil correcta: **dato siempre activo → CLAUDE.md · flujo → skill · garantía → hook**

Briefing de la tarea

Armar un toolkit SDD – skill + subagent + hook, para casa, en equipo

Tu tarea: un toolkit SDD

Andá más allá de un solo skill. Encodeá la disciplina SDD de las Lecciones 1-2 en un **toolkit** chico y funcional que el agente cargue solo.

Un skill

Encodeá un paso del pipeline que corriste a mano – p. ej. un **write-spec** brownfield que siempre exige alcance dentro/fuera + invariantes.

Un subagent

Un ayudante enfocado – p. ej. un agente **explorar** solo-lectura, o un agente **review-spec** independiente.

Un hook

Un guardrail que *garantiza* una práctica – p. ej. bloquear un commit si la suite de tests existente está roja.

Cada pieza tiene que encodear un **concepto de las Lecciones 1-2** – cobertura de VCs, alcance acotado, seguridad ante regresiones, o higiene de contexto.

El toolkit es el entregable; el punto es que la disciplina SDD ahora corre **sin que la retipees**.

Cómo encarar el toolkit

1 • Skill – encodeá un flujo

Agarrá un paso del pipeline que repetiste. Description en forma de trigger, pasos numerados, una plantilla, anti-patrones. Mapeá cada parte a un concepto SDD.

```
description: Usar cuando el usuario pide especificar un
cambio a código existente – exige alcance dentro/fuera +
invariantes.
```

2 • Subagent – delegá para enfocar

Escribí su brief: qué puede hacer (¿solo lectura?), qué devuelve (¿notas? ¿un veredicto?). Mostrá que mantiene limpio el contexto principal.

```
Solo explorar, sin editar. Devolvé módulos, interfaces y
riesgos como notas.
```

3 • Hook – garantizá un guardrail

Elegí un evento y un chequeo. Demostrá que dispara y que puede bloquear. Esta es una práctica que ya no podés saltar.

```
pre-commit: corré la suite existente; exit distinto de
cero ⇒ bloqueá el commit.
```

4 • (Opcional) Una rule

Fijá una restricción permanente en `AGENTS.md` – p. ej. que cada commit reference su spec + iteración.

5 • Probá que cada uno funciona

Dispará el skill, corré el subagent, hacé que el hook bloquee algo. Capturá evidencia de los tres.

Tarea (para antes de la Lección 4)

Trabajen en su **equipo de trabajo**. Conviertan la disciplina SDD de las Lecciones 1-2 en tooling que corre solo.



El toolkit

Un `SKILL.md`, un brief de subagent, y un hook – cada uno encodeando un concepto SDD con nombre



Evidencia de que funciona

El skill dispara, el subagent corre, el hook bloquea una acción mala – transcripciones o capturas



Un README corto

Qué hace cada pieza, qué concepto SDD encodea, cuándo la usan tus compañeros

Mové el playbook de tu cabeza al repo: **encodear → disparar → garantizar**

Entrega y presentación



Mail a la cátedra con el **link al repo** del equipo y el **hash del commit**



Hasta el **jueves al mediodía** de la semana anterior a la Lección 4



Al inicio de la Lección 4 se **sortea un equipo**: muestra su **hook bloqueando un commit malo**, en vivo



Miramos: el skill dispara solo, el subagent devuelve la conclusión, el hook veta, y el mapeo a L1-L2

Vengan todos listos para mostrar: el sorteo es en el momento. La consigna completa está en `examples/03-skills/tarea/enunciado.md`.

¿Preguntas?

Próxima: Lección 4 – MCP y plugins

Conectar SDD al ecosistema – tickets, docs y bases de datos.